Apostila de Apoio — Aula 4

Equivalência: ER → NFA → DFA

Teoria da Computação
Março/2026

Como usar

Esta apostila é para quem está começando do zero. A ideia aqui é bem prática: você já sabe descrever padrões com Expressões Regulares (ER) e já sabe que autômatos aceitam/rejeitam strings. Nesta aula, você vai aprender o caminho ER → NFA → DFA, usando exemplos pequenos e passo a passo.

Sumário

1. Visão geral
2. ER → NFA: construção por blocos (LEGO)
3. Exemplo completo ER → NFA (contém 01)
4. NFA → DFA: construção do conjunto de estados
5. Exemplo completo NFA → DFA (tabela)
6. Exercícios graduais
7. Erros comuns
8. Glossário rápido

1. Visão geral

Na Aula 2 você aprendeu Expressões Regulares (ER): uma forma compacta de descrever um conjunto de strings (uma linguagem). Na Aula 3 você aprendeu Autômatos (DFA e NFA): máquinas simples que leem uma string e decidem se ela é aceita.

Nesta aula, o objetivo é juntar as duas coisas:

A partir de uma ER, construir um NFA equivalente (uma máquina que reconhece a mesma linguagem).

A partir de um NFA, construir um DFA equivalente (sem escolhas, com um único caminho).

Resultado prático

Você ganha um "pipeline" de raciocínio: especificação em português → ER → NFA → DFA. Isso é a base teórica de como ferramentas e compiladores conseguem transformar descrições em máquinas eficientes.

2. ER → NFA: construção por blocos (LEGO)

A construção ER→NFA funciona como montar peças. Você não precisa memorizar uma prova; precisa apenas entender o sentido de cada operador da ER:

Símbolo (ex.: 0): aceita apenas a string "0".

União r|s: aceita o que r aceita OU o que s aceita.

Concatenação rs: aceita primeiro algo de r e depois algo de s.

Estrela r*: aceita repetir r zero ou mais vezes (inclui ε).

Observação importante (ε-transição)

Ao montar blocos, usamos a ideia de uma transição ε (épsilon): a máquina muda de estado sem ler símbolo nenhum. Isso serve apenas para “colar” peças. Você não vai programar isso: é uma ferramenta de desenho e raciocínio.

2.1 Bloco do símbolo (ex.: 0)

Para a ER 0, o NFA é apenas: início —(0)→ fim. Ele aceita somente a string "0".

2.2 Bloco da união (r | s)

A união vira uma escolha: do início, a máquina pode ir para o bloco de r ou para o bloco de s.

2.3 Bloco da concatenação (rs)

A concatenação vira uma ligação: o fim de r se conecta ao início de s (por ε).

2.4 Bloco da estrela (r*)

A estrela cria um “loop”: permite repetir o bloco r várias vezes, mas também permite sair sem repetir nenhuma vez (por isso ε é aceito).

3. Exemplo completo ER → NFA (contém 01)

Vamos usar uma ER muito comum, que descreve strings binárias que contêm a substring 01:

ER do exemplo
(0|1)*01(0|1)*

Interpretação em português:

Pode ter qualquer sequência de 0 e 1 antes.

Em algum momento aparece exatamente "01".

Depois pode ter qualquer sequência de 0 e 1.

Dividindo em partes (para montar por blocos):

A = (0|1)* (qualquer coisa antes)

B = 01 (o padrão obrigatório)

C = (0|1)* (qualquer coisa depois)

ER inteira = A B C (concatenação)

Como desenhar sem se perder
1) Desenhe o bloco do (0|1). 2) Transforme em estrela para ter (0|1)*. 3) Desenhe o bloco 0 seguido do bloco 1 para formar 01. 4) Concatene A, B e C ligando com ε.

4. NFA → DFA: construção do conjunto de estados

Um NFA pode ter escolhas: para um mesmo símbolo, ele pode seguir mais de um caminho. O DFA não pode escolher. Então o DFA guarda, como estado, um conjunto de estados possíveis do NFA.

Ideia central

Estado do DFA = conjunto de estados do NFA.
Aceita se o conjunto final contiver pelo menos um estado final do NFA.

4.1 Receita NFA → DFA (passo a passo)

1) Comece com o conjunto inicial {q0} (e os alcançáveis por ε, se existirem).

2) Para cada conjunto S e para cada símbolo (0 e 1), calcule para onde o NFA pode ir.

3) O resultado é um novo conjunto S'. Se ele ainda não existir como estado do DFA, crie.

4) Repita até não surgir conjunto novo.

5) Um conjunto é final no DFA se contiver um estado final do NFA.

5. Exemplo completo NFA → DFA (tabela) — contém 01

Usaremos o NFA didático da Aula 3, com estados q0 (início), q1 (vi 0 candidato) e q2 (final: encontrei 01).

Estado do NFA	lê 0	lê 1
q0	{q0,q1}	{q0}
q1	{q1}	{q2}
q2	{q2}	{q2}

Como ler a tabela acima

Exemplo: de q0 lendo 0, o NFA pode ficar em q0 (ignorar o 0) e ir para q1 (apostar que começa 01). Por isso o resultado é {q0, q1}.

Agora construímos o DFA com conjuntos. Começo com o estado inicial do DFA:

Estado inicial do DFA

{q0}

Tabela de construção do DFA (principais conjuntos):

Conjunto (estado do DFA)	lê 0	lê 1	É final?
{q0}	{q0,q1}	{q0}	não
{q0,q1}	{q0,q1}	{q0,q2}	não
{q0,q2}	{q0,q1,q2}	{q0,q2}	sim
{q0,q1,q2}	{q0,q1,q2}	{q0,q2}	sim

Conclusão do exemplo

O DFA acima reconhece exatamente a mesma linguagem do NFA: “contém 01”. A diferença é que agora não há escolhas: para cada conjunto e símbolo, existe um único próximo conjunto.

6. Exercícios graduais

Assuma Σ = {0,1}, a menos que o exercício diga o contrário.

Nível 1 — Entender a ER

1) Para a ER 0*, dê 4 strings aceitas e 2 rejeitadas.
2) Para a ER (0|1)*01, descreva em português o que ela aceita.

Nível 2 — ER → NFA por blocos

3) Monte um NFA para 0|1 e diga quais strings ele aceita.
4) Monte um NFA para 01 e diga quais strings ele aceita.

Nível 3 — NFA → DFA (conjuntos)

5) A partir do NFA “contém 01”, liste 4 conjuntos (estados do DFA) que aparecem.
6) Quais conjuntos são finais? Explique em uma frase.

7. Erros comuns

- Esquecer que r* inclui ε (zero repetições).
- Confundir 01 (concatenação) com 0|1 (união).
- No NFA→DFA, esquecer de fazer a união dos resultados quando o conjunto tem mais de um estado.
- No DFA de conjuntos, esquecer que um estado é final se contiver q2 (ou outro final do NFA).
- Misturar símbolos fora do alfabeto definido (Σ).

8. Glossário rápido

ER: expressão regular (descrição de uma linguagem regular).

NFA: autômato com escolhas (não determinístico).

DFA: autômato sem escolhas (determinístico).

ε-transição: transição sem ler símbolo (usada para ligar blocos).

Estado do DFA por conjuntos: um conjunto de estados possíveis do NFA.